Ejercicio Instituto

1. Insertar al menos 3 documentos de cada colección

```
use instituto

db.estudiantes.insertMany([
{ nombre: "Daniel Porras Morales", edad: 22, correo:
"[d](mailto:lucia.fernandez@correo.es)anielpm.dev@gmail.com", ciudad: "Tresjuncos" },
{ nombre: "Diego Martínez Ruiz", edad: 24, correo:
"[diego.martinez@gmail.com](mailto:diego.martinez@correo.es)", ciudad: "Valencia" },
{ nombre: "Carmen Torres López", edad: 21, correo:
"[carmen.torres@gmail.com](mailto:carmen.torres@correo.es)", ciudad: "Bilbao" }
])

db.cursos.insertMany([
{ nombre: "Sistemas de Big Data", codigo: "SBD101", creditos: 6, instructor: "Javier Peña" },
{ nombre: "Programacion de Inteligencia Artificial", codigo: "PIA202", creditos: 5, instructor:
"Laura Ortega" },
{ nombre: "Minería de Datos", codigo: "MD303", creditos: 4, instructor: "Andrés Muñoz" }
])

db.inscripciones.insertMany([
{ id_estudiante: ObjectId("68efd1e3e8759d8b66ce5f4b"), id_curso:
ObjectId("68efd1f3e8759d8b66ce5f4f"), anio: 2025, nota: 8.7 }, //Añadir el ID del objeto
creado
{ id_estudiante: ObjectId("68efd1e3e8759d8b66ce5f4d"), id_curso:
ObjectId("68efd1f3e8759d8b66ce5f4e"), anio: 2025, nota: 7.9 },
{ id_estudiante: ObjectId("68efd1e3e8759d8b66ce5f4c"), id_curso:
ObjectId("68efd1f3e8759d8b66ce5f50"), anio: 2025, nota: 9.1 }
])
```

```
serverSelectionTimeoutMS=2000
~
docker exec -it mongodb mongosh -u admin -p admin123

libreria>

libreria>

libreria>

libreria>

libreria>

libreria> use instituto
switched to db instituto
instituto>

instituto>

instituto>

instituto> db.estudiantes.insertMany([
... { nombre: "Daniel Porras Morales", edad: 22, correo: "danielpm.dev@gmail.com", ciudad: "Tresjuncos" },
... { nombre: "Diego Martinez Ruiz", edad: 24, correo: "diego.martinez@gmail.com", ciudad: "Valencia" },
... { nombre: "Carmen Torres López", edad: 21, correo: "carmen.torres@gmail.com", ciudad: "Bilbao" }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68efd1e3e8759d8b66ce5f4b'),
    '1': ObjectId('68efd1e3e8759d8b66ce5f4c'),
    '2': ObjectId('68efd1e3e8759d8b66ce5f4d')
  }
}
instituto>

instituto>

instituto> db.cursos.insertMany([
... { nombre: "Sistemas de Big Data", codigo: "SBD101", creditos: 6, instructor: "Javier Peña" },
... { nombre: "Programacion de Inteligencia Artificial", codigo: "PIA202", creditos: 5, instructor: "Laura Ortega" },
... { nombre: "Mineria de Datos", codigo: "MD303", creditos: 4, instructor: "Andrés Muñoz" }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68efd1f3e8759d8b66ce5f4e'),
    '1': ObjectId('68efd1f3e8759d8b66ce5f4f'),
    '2': ObjectId('68efd1f3e8759d8b66ce5f50')
  }
```

```
serverSelectionTimeoutMS=2000
~
docker exec -it mongodb mongosh -u admin -p admin123
(To exit, press Ctrl+C again or Ctrl+D or type .exit)
instituto> db.estudiantes.find().pretty()
[
  {
    _id: ObjectId('68efd1e3e8759d8b66ce5f4b'),
    nombre: 'Daniel Porras Morales',
    edad: 22,
    correo: 'danielpm.dev@gmail.com',
    ciudad: 'Tresjuncos'
  },
  {
    _id: ObjectId('68efd1e3e8759d8b66ce5f4c'),
    nombre: 'Diego Martinez Ruiz',
    edad: 24,
    correo: 'diego.martinez@gmail.com',
    ciudad: 'Valencia'
  },
  {
    _id: ObjectId('68efd1e3e8759d8b66ce5f4d'),
    nombre: 'Carmen Torres López',
    edad: 21,
    correo: 'carmen.torres@gmail.com',
    ciudad: 'Bilbao'
  }
]
instituto>
(To exit, press Ctrl+C again or Ctrl+D or type .exit)
instituto> db.inscripciones.insertMany([
... { id_estudiante: ObjectId("68efd1e3e8759d8b66ce5f4b"), id_curso: ObjectId("'68efd1f3e8759d8b66ce5f4f'"), anio: 2025, nota: 8.7 },
... { id_estudiante: ObjectId("68efd1e3e8759d8b66ce5f4d"), id_curso: ObjectId("68efd1f3e8759d8b66ce5f4e"), anio: 2025, nota: 7.9 },
... { id_estudiante: ObjectId("68efd1e3e8759d8b66ce5f4c"), id_curso: ObjectId("68efd1f3e8759d8b66ce5f50"), anio: 2025, nota: 9.1 }
... ])
BSONError: input must be a 24 character hex string, 12 byte Uint8Array, or an integer
instituto> db.inscripciones.insertMany([
... { id_estudiante: ObjectId("68efd1e3e8759d8b66ce5f4b"), id_curso: ObjectId("68efd1f3e8759d8b66ce5f4f"), anio: 2025, nota: 8.7 },
... { id_estudiante: ObjectId("68efd1e3e8759d8b66ce5f4d"), id_curso: ObjectId("68efd1f3e8759d8b66ce5f4e"), anio: 2025, nota: 7.9 },
... { id_estudiante: ObjectId("68efd1e3e8759d8b66ce5f4c"), id_curso: ObjectId("68efd1f3e8759d8b66ce5f50"), anio: 2025, nota: 9.1 }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68efd2cde8759d8b66ce5f51'),
    '1': ObjectId('68efd2cde8759d8b66ce5f52'),
    '2': ObjectId('68efd2cde8759d8b66ce5f53')
  }
}
instituto>
```

2. Buscar un curso por id

db.cursos.findOne({ _id: ObjectId("68efd1f3e8759d8b66ce5f4e") })

```
serverSelectionTimeoutMS=2000
~
docker exec -it mongodb mongosh -u admin -p admin123

Current Mongosh Log ID: 68efd4dc229feb5973ce5f46
Connecting to:          mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.8
Using MongoDB:          8.0.15
Using Mongosh:          2.5.8

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2025-10-15T16:23:53.753+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
   2025-10-15T16:23:54.439+00:00: Soft rlimits for open file descriptors too low
   2025-10-15T16:23:54.439+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
   2025-10-15T16:23:54.440+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
   2025-10-15T16:23:54.440+00:00: We suggest setting the contents of sysfsFile to 0.
------

test> use instituto
switched to db instituto
instituto>

instituto>

instituto>

instituto>

instituto>

instituto> db.estudiantes.find({ nombre: "Daniel Porras Morales" })
[
  {
    _id: ObjectId('68efd1e3e8759d8b66ce5f4b'),
    nombre: 'Daniel Porras Morales',
    edad: 22,
    correo: 'danielpm.dev@gmail.com',
    ciudad: 'Tresjuncos'
  }
]
instituto> db.cursos.findOne({ _id: ObjectId("68efd1f3e8759d8b66ce5f4e") })
{
  _id: ObjectId('68efd1f3e8759d8b66ce5f4e'),
  nombre: 'Sistemas de Big Data',
  codigo: 'SBD101',
  creditos: 6,
  instructor: 'Javier Peña'
}
instituto>
```

3. Buscar un alumno por nombre

db.estudiantes.find({ nombre: “Daniel Porras Morales” })

```
serverSelectionTimeoutMS=2000
~ (28.825s)
docker exec -it mongodb mongosh -u admin -p admin123

~ (0.023s)
clear

~
docker exec -it mongodb mongosh -u admin -p admin123

Current Mongosh Log ID: 68efd4dc229feb5973ce5f46
Connecting to:          mongodb://<credentials>@127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.8
Using MongoDB:          8.0.15
Using Mongosh:          2.5.8

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2025-10-15T16:23:53.753+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
   2025-10-15T16:23:54.439+00:00: Soft rlimits for open file descriptors too low
   2025-10-15T16:23:54.439+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
   2025-10-15T16:23:54.440+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
   2025-10-15T16:23:54.440+00:00: We suggest setting the contents of sysfsFile to 0.
------

test> use instituto
switched to db instituto
instituto>

instituto>

instituto>

instituto>

instituto>

instituto> db.estudiantes.find({ nombre: "Daniel Porras Morales" })
[
  {
    _id: ObjectId('68efd1e3e8759d8b66ce5f4b'),
    nombre: 'Daniel Porras Morales',
    edad: 22,
    correo: 'danielpm.dev@gmail.com',
    ciudad: 'Tresjuncos'
  }
]
instituto>
```

4. Actualizar la ciudad de un estudiante

db.estudiantes.updateOne(
{ nombre: "Diego Martínez Ruiz" },
{ $set: { ciudad: "Ciudad Real" } }
)

```
serverSelectionTimeoutMS=2000
~
docker exec -it mongodb mongosh -u admin -p admin123
instituto>
instituto>
instituto>
instituto>
instituto>
instituto>
instituto>
instituto>
instituto>
instituto>
instituto>
instituto>

instituto> db.estudiantes.updateOne(
... { nombre: "Diego Martínez Ruiz" },
... { $set: { ciudad: "Ciudad Real" } }
... )
...
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
instituto> db.estudiantes.find({ nombre: "Diego Martínez Ruiz" })
[
  {
    _id: ObjectId('68efd1e3e8759d8b66ce5f4c'),
    nombre: 'Diego Martínez Ruiz',
    edad: 24,
    correo: 'diego.martinez@gmail.com',
    ciudad: 'Ciudad Real'
  }
]
instituto>
```

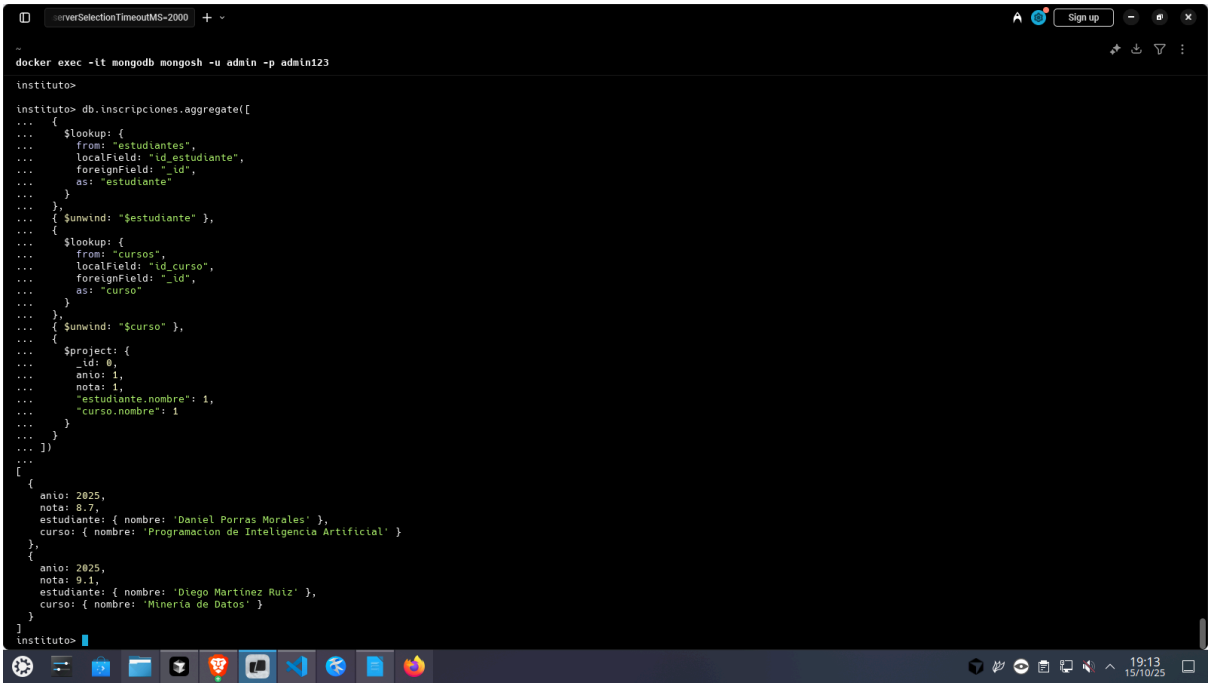
5. Eliminar una inscripción

db.inscripciones.deleteOne({ id_estudiante: ObjectId("68efd1e3e8759d8b66ce5f4d") })

```
serverSelectionTimeoutMS=2000
~
docker exec -it mongodb mongosh -u admin -p admin123
instituto>
instituto>
instituto>
instituto>
instituto>
instituto>
instituto>
instituto>
instituto>

instituto> db.estudiantes.updateOne(
... { nombre: "Diego Martínez Ruiz" },
... { $set: { ciudad: "Ciudad Real" } }
... )
...
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
instituto> db.estudiantes.find({ nombre: "Diego Martínez Ruiz" })
[
  {
    _id: ObjectId('68efd1e3e8759d8b66ce5f4c'),
    nombre: 'Diego Martínez Ruiz',
    edad: 24,
    correo: 'diego.martinez@gmail.com',
    ciudad: 'Ciudad Real'
  }
]
instituto>

instituto>

instituto> db.inscripciones.deleteOne({ id_estudiante: ObjectId("68efd1e3e8759d8b66ce5f4d") })
{ acknowledged: true, deletedCount: 1 }
instituto>
```

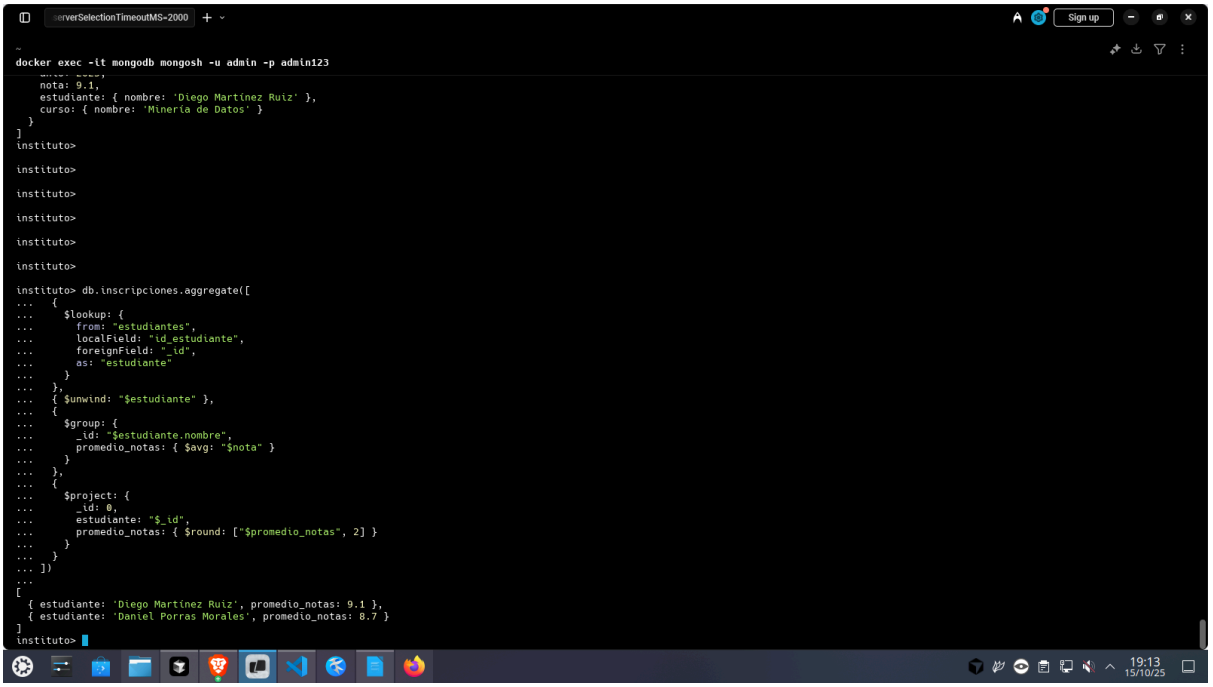
6. Agregación. Mostrar las inscripciones con nombres de estudiantes y asignaturas. Es decir, cada documento resultante debe contener todos los campos de una inscripción además del nombre del alumno y de la asignatura.

```
db.inscripciones.aggregate([
  {
    $lookup: {
      from: "estudiantes",
      localField: "id_estudiante",
      foreignField: "_id",
      as: "estudiante"
    }
  },
  { $unwind: "$estudiante" },
  {
    $lookup: {
      from: "cursos",
      localField: "id_curso",
      foreignField: "_id",
      as: "curso"
    }
  },
  { $unwind: "$curso" },
  {
    $project: {
      _id: 0,
      anio: 1,
      nota: 1,
      "estudiante.nombre": 1,
      "curso.nombre": 1
    }
  }
])
```



7. Agregación. Calcular el promedio de notas por estudiante

```
db.inscripciones.aggregate([
  {
    $lookup: {
      from: "estudiantes",
      localField: "id_estudiante",
      foreignField: "_id",
      as: "estudiante"
    }
  },
  { $unwind: "$estudiante" },
  {
    $group: {
      _id: "$estudiante.nombre",
      promedio_notas: { $avg: "$nota" }
    }
  },
  {
    $project: {
      _id: 0,
      estudiante: "$_id",
      promedio_notas: { $round: ["$promedio_notas", 2] }
    }
  }
])
```



8. Agregación. Contar cuántos estudiantes hay inscritos por asignatura.

```
db.inscripciones.aggregate([
  {
    $lookup: {
      from: "cursos",
      localField: "id_curso",
      foreignField: "_id",
      as: "curso"
    }
  },
  { $unwind: "$curso" },
  {
    $group: {
      _id: "$curso.nombre",
      total_estudiantes: { $sum: 1 }
    }
  },
  {
    $project: {
      _id: 0,
      curso: "$_id",
      total_estudiantes: 1
    }
  }
])
```

serverSelectionTimeoutMS=2000

Sign up

```
~
docker exec -it mongodb mongosh -u admin -p admin123
  { estudiante: 'Daniel Porras Morales', promedio_notas: 8.7 }
]
instituto>

instituto>

instituto>

instituto>

instituto>

instituto>

instituto> db.inscripciones.aggregate([
...   {
...     $lookup: {
...       from: "cursos",
...       localField: "id_curso",
...       foreignField: "_id",
...       as: "curso"
...     }
...   },
...   { $unwind: "$curso" },
...   {
...     $group: {
...       _id: "$curso.nombre",
...       total_estudiantes: { $sum: 1 }
...     }
...   },
...   {
...     $project: {
...       _id: 0,
...       curso: "$_id",
...       total_estudiantes: 1
...     }
...   }
... ])
...
[
  { total_estudiantes: 1, curso: 'Mineria de Datos' },
  {
    total_estudiantes: 1,
    curso: 'Programacion de Inteligencia Artificial'
  }
]
instituto>
```

19:14
15/10/25